SkillBridge – Backend Requirements Document (Node.js Version – V1)

Core idea :

“ SkillBridge is a subscription-based maintenance management platform for housing societies. Instead of residents individually searching for plumbers or electricians, the entire society subscribes to a structured system where residents can raise maintenance tickets, verified workers are assigned, work is verified through OTP, and every job is recorded digitally with photos and history. The platform focuses on accountability, transparency, and organized maintenance operations at the society level. “

Requirements :

(Verify with what i said u on phone and verify if any doubt then contact , minor changes will be there try to fix if not done then ask me, for now consider this name only fixed will be provided later) !!

1. Project Overview

Product Name: SkillBridge

Product Type: B2B Housing Society Maintenance Management System

Recommended Backend Stack

- Node.js (LTS version)
- Express.js (or Fastify)
- PostgreSQL
- Redis (OTP & caching)
- JWT Authentication
- Multer (file uploads)
- Bull / Agenda (background jobs)
- AWS EC2 + Nginx + PM2 for deployment

2. User Roles

1. Admin
2. Resident (Flat Owner)
3. Worker

Role-based access control must be enforced at middleware level.

3. Society Management

Admin Capabilities:

- Create Society
- Update Society
- Activate/Deactivate Subscription
- Open/Close Registration Window

Society Fields:

- id (UUID)
- name
- address
- pincode
- total_flats
- subscription_status (active/inactive)
- registration_open (boolean)
- created_at
- updated_at

4. Flat Bulk Upload

Admin uploads CSV containing flat_number and tower (optional).

"this is optional put ,what we have discussed i.e user will visit and egister himself accordding to the website."

System Requirements:

- Validate duplicates
- Insert flats linked to society
- Set is_registered = false

Flat Fields:

- id (UUID)
- society_id (FK)
- tower
- flat_number
- is_registered (boolean)
- created_at

Unique constraint: (society_id + flat_number)

5. Resident Registration

- Society.registration_open must be true
- Flat must exist
- Flat.is_registered must be false
- Mobile must not already be linked

After OTP verification:

- Create User
- Assign role = resident
- Link user to flat
- Mark flat.is_registered = true

6. Authentication

- OTP-based authentication
- 6-digit numeric OTP
- Stored in Redis (expiry 5 minutes)
- Max 3 attempts
- Return JWT access + refresh tokens

7. Worker Management

- Create worker
- Assign skill type (plumbing/electrical)
- Assign worker to societies
- Activate/Deactivate worker

Worker Fields:

- Id (user can see that while booking)
- Name (user can see that while booking)
- Adhar card, pancard , DOB (to register)
- Phone (user can see that while booking)
- skill_type (user can see that while booking)
- is_active (user can see that while booking)
- created_at (user can see that while booking)

8. Ticket System

Resident Ticket Creation:

- category (plumbing/electrical)
- description
- optional image

System Actions:

- Validate subscription active
- Create ticket (status = OPEN)
- Auto-assign worker
- Notify worker

Ticket Status Lifecycle:

- OPEN
- ASSIGNED
- ACCEPTED
- IN_PROGRESS
- COMPLETED
- CANCELLED

9. Worker Flow

- View assigned tickets
- Accept/Reject ticket
- Mark Arrived (generate Start OTP)
- Enter Start OTP (status = IN_PROGRESS)
- Upload after-work photo (mandatory)
- Upload part photo (optional)
- Mark job COMPLETE

10. Review & Grievance

Review:

- Rating (1–5)
- Comment
- One review per ticket

Grievance:

- Linked to ticket
- Status: OPEN / IN_REVIEW / RESOLVED / REJECTED

11. Service History

- List of all past tickets
- Category
- Worker name
- Photos
- Rating
- Status

12. Contact Us

- User ID (nullable)
- Subject
- Message

13. Security Requirements

- JWT middleware
- Role-based route guards

- Input validation (Joi/Zod)
- Rate limiting on OTP endpoints
- File upload size limit
- Secure S3 storage for images

14. Non-Functional Requirements

- Support 10–20 societies initially
- Handle ~100 tickets/day
- UTC timestamps
- Centralized logging
- Global error handling middleware

15. Future Scope (Not in V1)

- Payment integration
- Premium services
- Live GPS tracking
- Multi-user per flat
- Mobile application
- Analytics dashboard